SET-5:

1. A computer vision lab is processing small grayscale image patches represented as 2D numerical matrices. Develop Python programs using NumPy for the following requirements:

(a) Extract a specific sub-region (slice) from a given 4×4 image matrix by selecting rows 1 to 2 and columns 1 to 2.

(b) Reshape a flat 1D pixel stream of 9 elements into a 3×3 image patch, and then amplify its pixel intensities by computing the cube of all elements.

(c) Build a composite 4×4 test pattern by creating a block matrix consisting of a 2×2 Identity matrix, a 2×2 Zero matrix, and two constant 2×2 matrices.

Solution:

```
# a) Matrix Slicing
import numpy as np
image_patch = np.array(
   [
      [10, 20, 30, 40],
      [50, 60, 70, 80],
      [90, 100, 110, 120],
      [130, 140, 150, 160],
   ]
)
# Extract rows 1 to 2 and columns 1 to 2 (0-indexed: index 1 and 2)
sub_region = image_patch[1:3, 1:3]
print("(a) Extracted 2x2 Sub-Region:\n", sub_region)
# b) Reshaping and Cube
pixel_stream = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9])
reshaped_patch = pixel_stream.reshape(3, 3)
cubed_patch = reshaped_patch**3
print("\n(b) Reshaped 3x3 Matrix:\n", reshaped_patch)
print("    Cubed Patch:\n", cubed_patch)
# c) Block Matrix using Identity and Zero Matrix
I = np.eye(2, dtype=int)  # 2x2 Identity Matrix
Z = np.zeros((2, 2), dtype=int)  # 2x2 Zero Matrix
A = np.full((2, 2), 5)  # 2x2 Matrix with constant 5
```

```
B = np.full((2, 2), 9)  # 2x2 Matrix with constant 9
block_matrix = np.block([[I, Z], [A, B]])
print("\n(c) 4x4 Composite Block Matrix:\n", block_matrix)
```

Output:

```
(a) Extracted 2x2 Sub-Region:
 [[ 60  70]
 [100 110]]
(b) Reshaped 3x3 Matrix:
 [[1 2 3]
 [4 5 6]
 [7 8 9]]
   Cubed Patch:
 [[  1   8  27]
 [ 64 125 216]
 [343 512 729]]
(c) 4x4 Composite Block Matrix:
 [[1 0 0 0]
 [0 1 0 0]
 [5 5 9 9]
 [5 5 9 9]]
```

2. A smart monitoring system records continuous temperature readings and receives encrypted device tokens over a network. Develop Python programs for the following requirements:

(a) A sequence of temperature readings is recorded during a test run. Write a program to find the length of the longest continuous period where temperature readings strictly increase from one observation to the next.

(b) A transmission sends two device identification codes. Determine whether one code is a circular rotation of the other code.

Solution:

```
# --- (a) Longest Increasing Continuous Sublist ---
def longest_increasing_sublist(arr):
    if not arr:
        return 0
```

```
    max_len = 1
    curr_len = 1
    for i in range(1, len(arr)):
        if arr[i] > arr[i - 1]:
            curr_len += 1
            max_len = max(max_len, curr_len)
        else:
            curr_len = 1
    return max_len
readings = [12, 15, 18, 14, 16, 20, 22, 19]
print("(a) Longest Increasing Sublist Length:", longest_increasing_sublist(readings))
# --- (b) Check String Rotation ---
def is_rotation(s1, s2):
    if len(s1) != len(s2) or not s1:
        return False
    return s2 in (s1 + s1)
token1 = "ABCD"
token2 = "CDAB"
print("(b) Is Rotation:", is_rotation(token1, token2))
```

Output:

(a) Longest Increasing Sublist Length: 4

(b) Is Rotation: True

3. A robotics laboratory tracks autonomous robot sensor diagnostics and manages monthly spare-part billing across four robotic workgroups. Develop suitable Python programs using NumPy and generator expressions for the following requirements:

(a) A robot logs 1000 continuous sensor readings generated via a lazy generator expression. Standardize the data using its sample mean and standard deviation, and verify that the normalized dataset has zero mean and unit standard deviation.

(b) The facility stocks three types of spare parts for four robot workgroups. Given a 4×3 base cost matrix S, a 1D discount vector d of length 3, and a 2D tax vector t of shape 4×1, use NumPy broadcasting to compute the final billed amounts and determine which workgroup generated the maximum total revenue.

Solution:

```
import numpy as np
# (a) Random Sensor Readings and Normalization
data = np.fromiter((np.random.rand() for _ in range(1000)), dtype=float)
mean = np.mean(data)
std = np.std(data)
normalized = (data - mean) / std
print("--- (a) Sensor Normalization ---")
print("Original Mean =", mean)
print("Original Standard Deviation =", std)
print("Normalized Data =", normalized[:5], "...")
print("Normalized Mean =", np.mean(normalized))
print("Normalized Standard Deviation =", np.std(normalized))
print("Is mean close to 0?:", np.isclose(np.mean(normalized), 0))
# (b) Broadcasting - Robot Spare-Part Billing
S = np.array([
    [320, 360, 400],
    [280, 310, 350],
    [450, 480, 520],
    [240, 270, 300]
])
d = np.array([20, 25, 30])
t = np.array([[0.05], [0.08], [0.10], [0.06]])
final = (S - d) * (1 + t)
revenue = final.sum(axis=1)
best = np.argmax(revenue)
print("\n--- (b) Robot Spare-Part Billing ---")
print("Final Bill Matrix Shape:", final.shape)
print("Final Bill Matrix:\n", final)
print("Revenue by Group:", revenue)
print("Maximum Revenue Group:", best + 1)
```

Output:

```
--- (a) Sensor Normalization ---
```

Original Mean = 0.4903808310647585

Original Standard Deviation = 0.29485272358691794

Normalized Data = [-1.4017097 1.22485802 0.18941938 -1.06562163 0.03414541] ...

Normalized Mean = -1.7763568394002505e-17

Normalized Standard Deviation = 0.9999999999999999

Is mean close to 0?: True

--- (b) Robot Spare-Part Billing ---

Final Bill Matrix Shape: (4, 3)

Final Bill Matrix:

[[315. 351.75 388.5]

[280.8 307.8 345.6]

[473. 500.5 539.]

[233.2 259.7 286.2]]

Revenue by Group: [1055.25 934.2 1512.5 779.1]

Maximum Revenue Group: 3

SET-6:

1. An examination department processes a list of raw student assessment scores to compile rank reports. Develop suitable Python programs for the following requirements:

(a) A test score list contains multiple identical marks. Eliminate all duplicate entries and arrange the unique scores from the lowest to the highest.

(b) To mitigate grading anomalies, eliminate the lowest score and the highest score from a list of marks, and then display the remaining values in reverse order of entry.

(c) Create a separate duplicate of the original marks list and sort it in descending order to publish a merit list while ensuring the original list remains unaltered.

Solution:

```
# (a) Remove Duplicates and Sort Ascending
raw_scores = [75, 42, 88, 75, 60, 42, 95, 88]
# Convert to set to eliminate duplicates, then sort in ascending order
unique_sorted_scores = sorted(list(set(raw_scores)))
print("--- (a) Unique Scores (Ascending) ---")
print("Original Scores:", raw_scores)
print("Processed Scores:", unique_sorted_scores)
```

(b) Remove Smallest and Largest, Then Reverse

```
marks = [55, 12, 98, 45, 82, 67]
print("\n--- (b) Trim Extremes and Reverse ---")
print("Original Marks:", marks)
# Remove the minimum and maximum elements
marks.remove(min(marks))
marks.remove(max(marks))
# Reverse the remaining elements in place
marks.reverse()
print("Remaining Marks (Reversed):", marks)
```

(c) Copy and Sort in Descending Order

```
base_marks = [68, 92, 45, 85, 77]
# Create a shallow copy so the original sequence remains preserved
merit_list = base_marks.copy()
merit_list.sort(reverse=True)
print("\n--- (c) Copy and Sort Descending ---")
print("Original List (Unchanged):", base_marks)
print("Merit List (Descending):", merit_list)
```

Output:

```
--- (a) Unique Scores (Ascending) ---
Original Scores: [75, 42, 88, 75, 60, 42, 95, 88]
Processed Scores: [42, 60, 75, 88, 95]

--- (b) Trim Extremes and Reverse ---
Original Marks: [55, 12, 98, 45, 82, 67]
Remaining Marks (Reversed): [67, 82, 45, 55]

--- (c) Copy and Sort Descending ---
Original List (Unchanged): [68, 92, 45, 85, 77]
Merit List (Descending): [92, 85, 77, 68, 45]
```

2. A quality inspection team on a manufacturing floor logs test metrics for manufactured parts to verify variations across a production batch. Develop suitable Python programs using NumPy and Matplotlib for the following requirements:

(a) Generate 100 random integer measurement readings ranging between 1 and 100 to represent component stress values. Draw a simple line plot connecting all 100 measurements against their sample indices (1 to 100).

(b) Using the same 100 generated measurement readings, create a scatter plot displaying individual data points to highlight the dispersion of the metrics.

(c) Represent the identical 100 random readings using both a vertical bar chart and a horizontal bar chart with suitable axis titles and legends.

Solution:

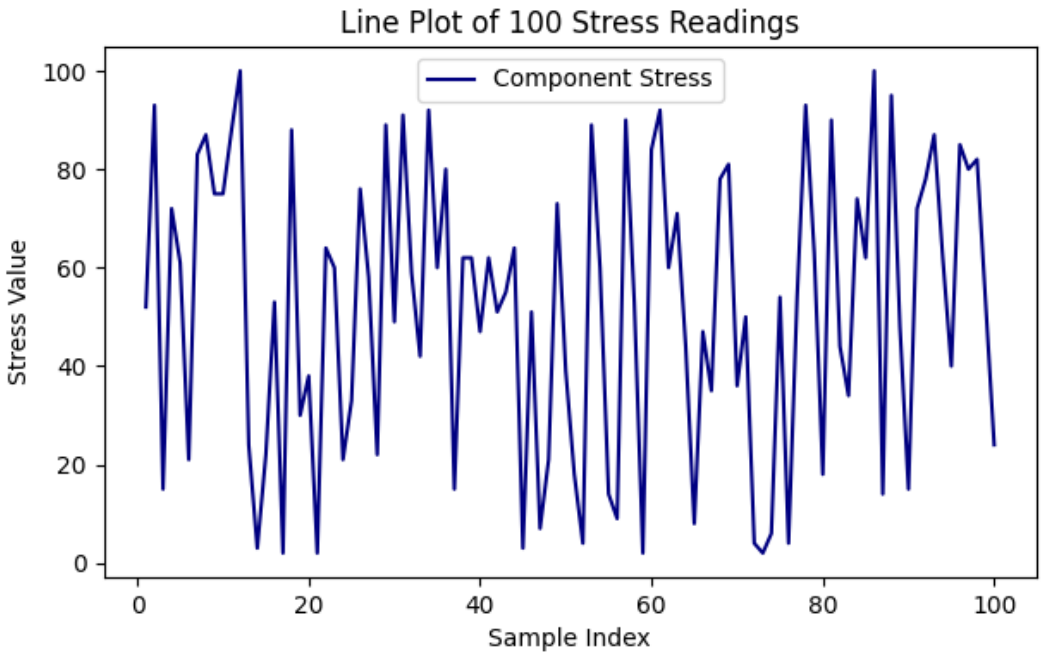
```
import numpy as np
import matplotlib.pyplot as plt
# Generate 100 random integer readings (1 to 100)
np.random.seed(42)
measurements = np.random.randint(1, 101, size=100)
sample_index = np.arange(1, 101)
# (a) Line Plot of 100 Measurements
plt.figure(figsize=(7, 4))
plt.plot(sample_index, measurements, color='navy', label='Component Stress')
plt.title('Line Plot of 100 Stress Readings')
plt.xlabel('Sample Index')
plt.ylabel('Stress Value')
plt.legend()
plt.show()
# (b) Scatter Plot of 100 Measurements
plt.figure(figsize=(7, 4))
plt.scatter(sample_index, measurements, color='crimson', label='Inspection Data')
plt.title('Scatter Plot of 100 Stress Readings')
plt.xlabel('Sample Index')
plt.ylabel('Stress Value')
plt.legend()
plt.show()
```

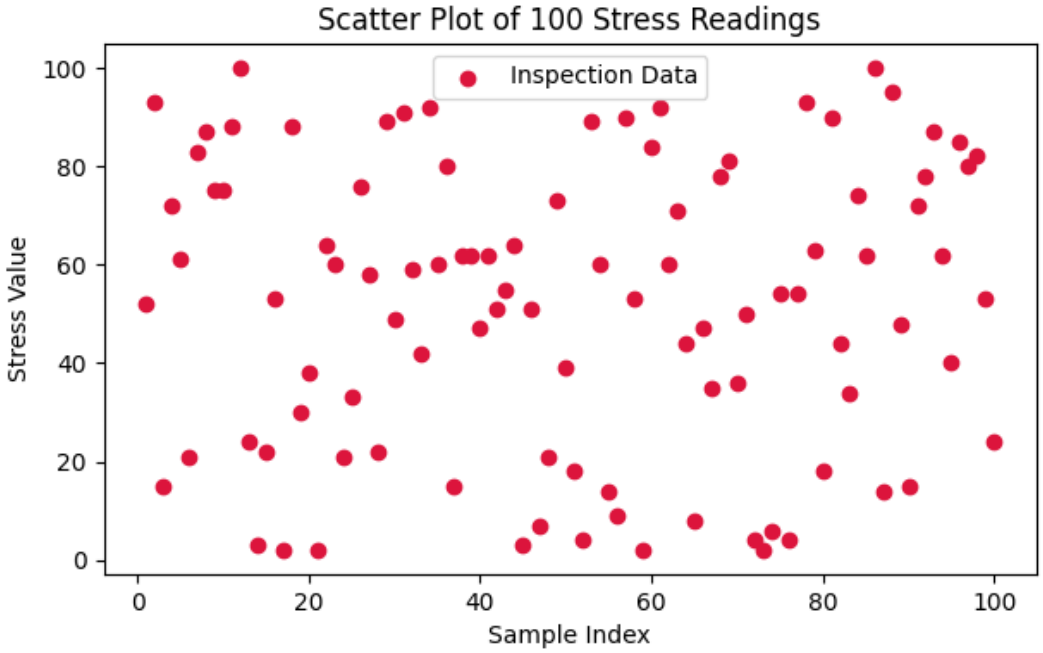
(c) Vertical and Horizontal Bar Plots

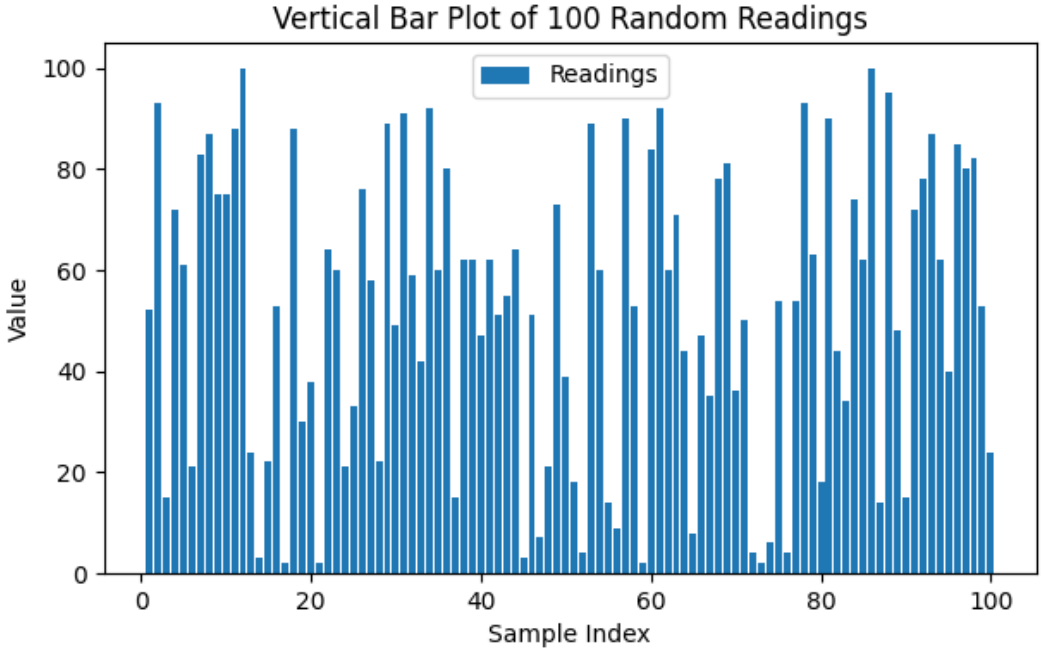
```
plt.figure(figsize=(7, 4))
plt.bar(sample_index, measurements, label='Readings')
plt.title('Vertical Bar Plot of 100 Random Readings')
plt.xlabel('Sample Index')
plt.ylabel('Value')
plt.legend()
plt.show()

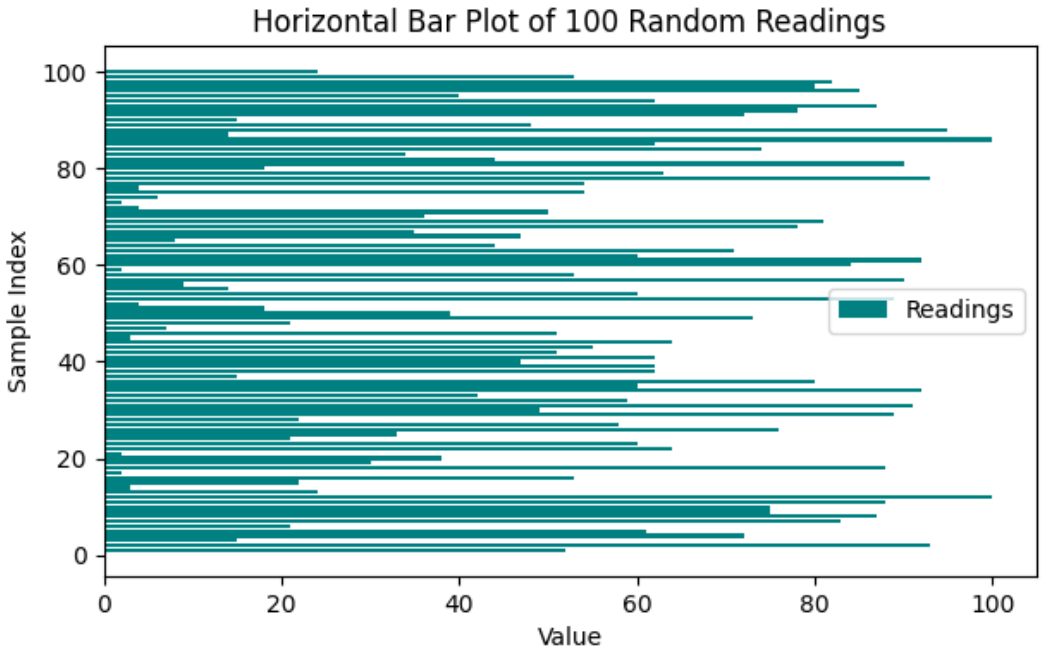
plt.figure(figsize=(7, 4))
plt.barh(sample_index, measurements, color='teal', label='Readings')
plt.title('Horizontal Bar Plot of 100 Random Readings')
plt.xlabel('Value')
plt.ylabel('Sample Index')
plt.legend()
plt.show()
```

Output:




Scatter Plot of 100 Stress Readings
Inspection Data
Stress Value
Sample Index
0
20
40
60
80
100


Vertical Bar Plot of 100 Random Readings
Readings
Value
Sample Index
0
20
40
60
80
100



3. An online retail platform records monthly sales units for various electronic brands to evaluate catalog performance. Develop suitable Python programs for the following requirements:

a) The sales units for four smartphone brands are stored in a dictionary. Find and display the total sales units across all brands.

b) From the brand sales record, filter and display only those brands that have sold more than 50 units using dictionary comprehension.

c) Identify and display the specific brand that recorded the maximum sales volume during the month.

Solution:

```
# Monthly sales units for 4 brands
brand_sales = {"Samsung": 45, "Apple": 82, "OnePlus": 95, "Xiaomi": 38}
print("--- Initial Brand Sales Data ---")
print(brand_sales)
# (a) Sum All Values in a Dictionary
total_units = sum(brand_sales.values())
print("\n(a) Total sales units sold across all brands:", total_units)
# (b) Filter Dictionary Items by Value (Comprehension)
high_performing = {brand: sales for brand, sales in brand_sales.items() if sales > 50}
print("\n(b) Brands with sales greater than 50 units:")
print(high_performing)
```

(c) Find Key with Maximum Value

```
top_brand = max(brand_sales, key=brand_sales.get)
print(f"\n(c) Top-selling brand: {top_brand} (Units: {brand_sales[top_brand]})")
```

Output:

```
--- Initial Brand Sales Data ---
{'Samsung': 45, 'Apple': 82, 'OnePlus': 95, 'Xiaomi': 38}

(a) Total sales units sold across all brands: 260

(b) Brands with sales greater than 50 units:
{'Apple': 82, 'OnePlus': 95}

(c) Top-selling brand: OnePlus (Units: 95)
```

SET-7:

1. An examination committee is processing student test results to generate automated performance reports. Develop suitable Python programs for the following requirements:

a) Accept a list of marks obtained by a group of students and compute the average score of the batch.

(b) Define a user-defined function that accepts the marks list and returns the highest score obtained among all students.

(c) Inspect the list of student marks and determine how many students received an even numerical mark.

Solution:

(a) Average Marks of Students

```
marks = list(map(float, input("Enter marks separated by spaces: ").split()))
average = sum(marks) / len(marks)
print("Average Marks:", round(average, 2))
```

(b) User-Defined Function to Return Maximum Element

```
def max_element(lst):
    return max(lst)
print("Maximum Score:", max_element(marks))
```

(c) Count Even Numbers in a List

```
even_count = sum(1 for m in marks if int(m) % 2 == 0)
```

```
print("Count of Even Marks:", even_count)
```

Output:

```
Enter marks separated by spaces: 75 80 44 55 60
Average Marks: 62.8
Maximum Score: 80.0
Count of Even Marks: 3
```

2. A stochastic computing laboratory runs Monte-Carlo approximations and evaluates streaming sensor data buffers. Develop suitable Python programs using NumPy and generator expressions for the following requirements:

(a) Formulate a generator of 10,000 coordinate pairs in the unit square $[0, 1] \times [0, 1]$ and compute the Monte-Carlo estimate of the mathematical constant π based on the points falling inside the quarter circle.

(b) Synthesize a continuous stream of 200 observation readings. Generate sliding temporal windows of length 10 using a generator expression and compute the sample covariance for each successive window.

(c) Generate 500 random sensor values lazily using np.fromiter and compute the statistical mean, variance, and median of the dataset.

Solution:

```
import numpy as np
# (a) Monte-Carlo Estimation of Pi
N = 10000
points = ((np.random.rand(), np.random.rand()) for _ in range(N))
inside = sum(1 for x, y in points if x * x + y * y <= 1)
pi_est = 4 * inside / N
print("--- (a) Monte-Carlo Estimation of Pi ---")
print("Total Points Inside Quarter Circle:", inside)
print("Estimated Pi:", pi_est)
# (b) Sliding Windows and Covariance
data_stream = np.random.rand(200)
windows = (data_stream[i : i + 10] for i in range(len(data_stream) - 9))
covariances = [np.cov(w) for w in windows]
print("\n--- (b) Sliding Windows and Covariance ---")
print("Total Windows Evaluated:", len(covariances))
```

```
print("First 3 Window Covariances:\n", covariances[:3])
```

(c) Mean, Variance and Median of 500 Random Values

```
random_vals = np.fromiter((np.random.rand() for _ in range(500)), dtype=float)
print("\n--- (c) Statistics of 500 Random Values ---")
print("Mean:", np.mean(random_vals))
print("Variance:", np.var(random_vals))
print("Median:", np.median(random_vals))
```

Output:

```
--- (a) Monte-Carlo Estimation of Pi ---
Total Points Inside Quarter Circle: 7825
Estimated Pi: 3.13
--- (b) Sliding Windows and Covariance ---
Total Windows Evaluated: 191
First 3 Window Covariances:
 [array(0.0663432), array(0.02729099), array(0.05091509)]
--- (c) Statistics of 500 Random Values ---
Mean: 0.49234006911840006
Variance: 0.08621641649701599
Median: 0.4775325075497675
```

3. A document processing pipeline cleans text entries and generates shorthand code representations for sensitive files. Develop suitable Python programs using list comprehensions for the following requirements:

(a) An analyst provides a list of mixed tokens containing punctuation and alphanumeric codes. Extract only the purely alphabetic words and convert them to lowercase using a list comprehension.

(b) In an input sentence, locate all words having a length strictly greater than 4 characters and reverse each of them while leaving other words out.

(c) Given a list of descriptive tags, create two-letter abbreviations for every word having at least 2 characters by combining its first and last characters.

Solution:

(a) Keep Alphabetic Words in Lowercase

```
tokens = ["DataScience", "AI#2026", "Machine", "NLP!", "analytics", "100x"]
```

```
clean_words = [t.lower() for t in tokens if t.isalpha()]
print("(a) Alphabetic Words in Lowercase:", clean_words)
# (b) Reverse Words Longer Than Four Characters
sentence = "cloud systems deliver high performance output"
reversed_words = [w[::-1] for w in sentence.split() if len(w) > 4]
print("(b) Reversed Words (> 4 chars):", reversed_words)
# (c) First-and-Last-Letter Abbreviation
tags = ["router", "ip", "s", "gateway", "switch"]
abbreviations = [w[0] + w[-1] for w in tags if len(w) >= 2]
print("(c) Two-Letter Abbreviations:", abbreviations)
```

Output:

(a) Alphabetic Words in Lowercase: ['datascience', 'machine', 'analytics']

(b) Reversed Words (> 4 chars): ['duolc', 'smetsys', 'reviled', 'ecnamrofrep', 'tuptuo']

(c) Two-Letter Abbreviations: ['rr', 'ip', 'gy', 'sh']

SET-8:

1. A university placement cell compiles hiring statistics across multiple academic branches and salary offerings to observe historical trends. Develop suitable Python programs using Matplotlib for the following requirements:

(a) Five academic years (1 to 5) are monitored. In Series 1, hiring counts are recorded as [40, 48, 55, 62, 70], and in Series 2, they are recorded as [35, 42, 45, 50, 58]. Draw a multiple line plot displaying both series on the same chart with circular and square markers and a clear legend

(b) A student budget breakdown allocates funds as Tuition (45%), Rent (25%), Food (18%), and Books (12%). Construct a pie chart highlighting the percentage share of each category.

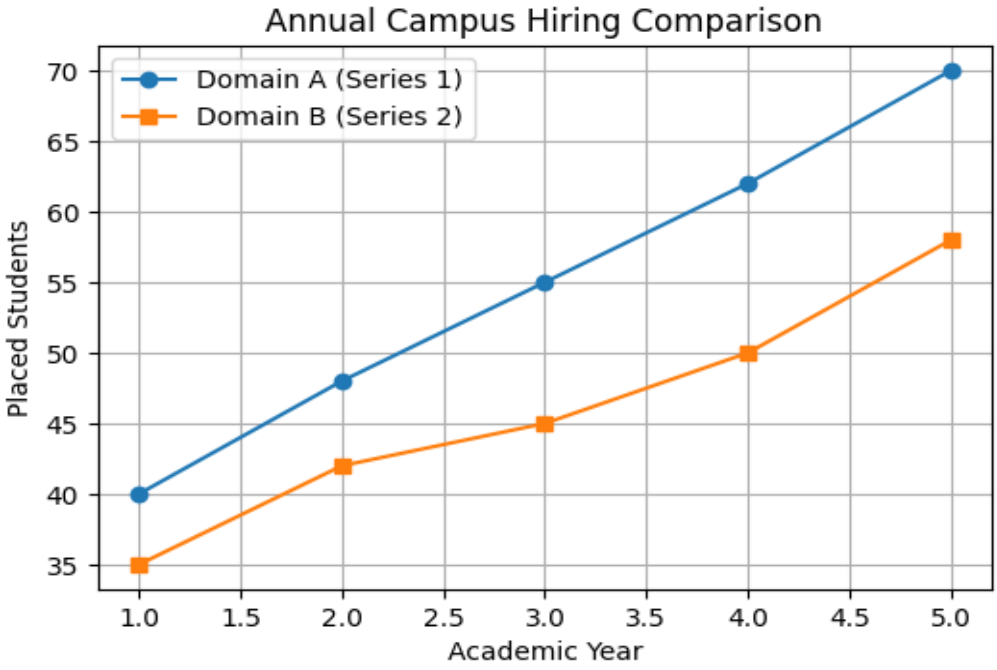
(c) Five departments (A, B, C, D, E) record company visits of 18, 32, 24, 28, and 15 respectively. Construct a bar chart showing these values with proper labels and an informative title.

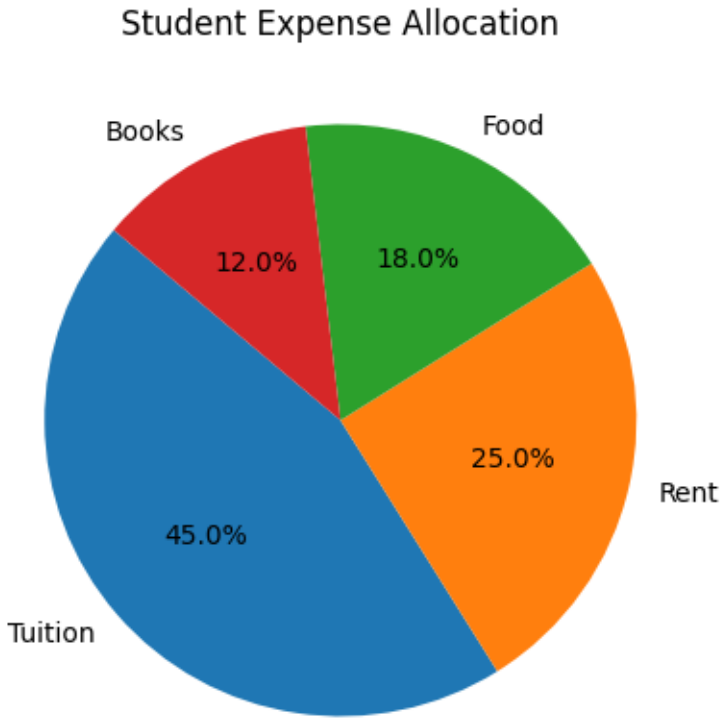
Solution:

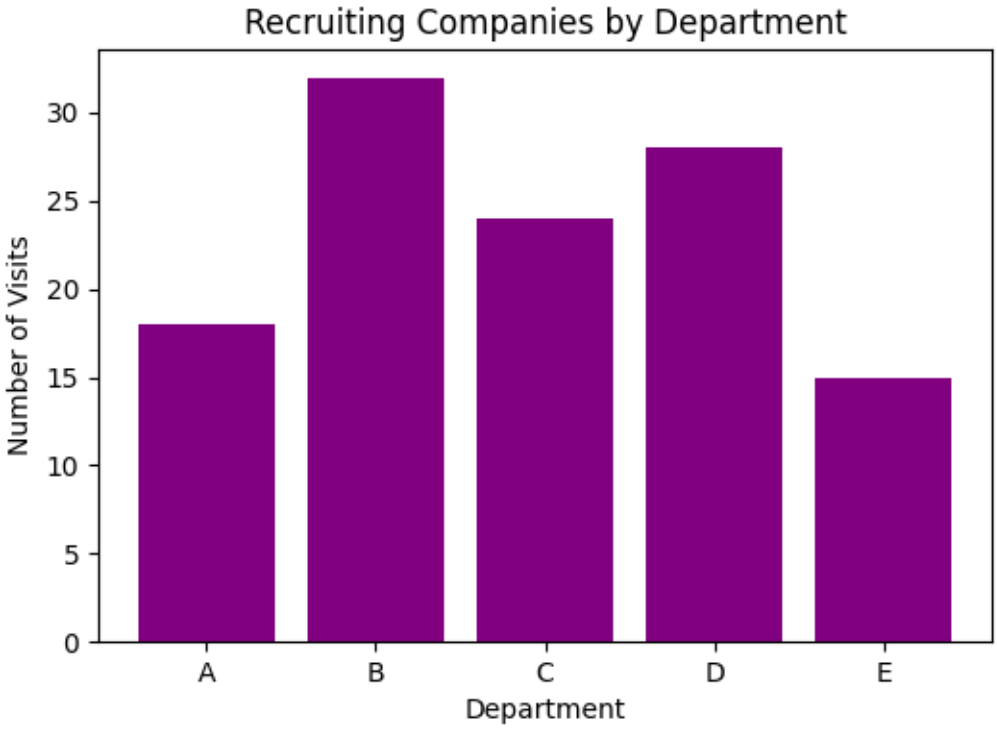
```
import matplotlib.pyplot as plt
# (a) Multiple Line Plot: Multi-Year Hiring Trends
years = [1, 2, 3, 4, 5]
series_1 = [40, 48, 55, 62, 70]
series_2 = [35, 42, 45, 50, 58]
plt.figure(figsize=(6, 4))
```

```
plt.plot(years, series_1, marker='o', label='Domain A (Series 1)')
plt.plot(years, series_2, marker='s', label='Domain B (Series 2)')
plt.title('Annual Campus Hiring Comparison')
plt.xlabel('Academic Year')
plt.ylabel('Placed Students')
plt.legend()
plt.grid(True)
plt.show()
# (b) Pie Chart: Student Budget Allocation
expenditures = ['Tuition', 'Rent', 'Food', 'Books']
proportions = [45, 25, 18, 12]
plt.figure(figsize=(5, 5))
plt.pie(proportions, labels=expenditures, autopct='%1.1f%%', startangle=140)
plt.title('Student Expense Allocation')
plt.show()
# (c) Bar Chart: Department Visits
depts = ['A', 'B', 'C', 'D', 'E']
company_visits = [18, 32, 24, 28, 15]
plt.figure(figsize=(6, 4))
plt.bar(depts, company_visits, color='purple')
plt.title('Recruiting Companies by Department')
plt.xlabel('Department')
plt.ylabel('Number of Visits')
plt.show()
```

Output


Annual Campus Hiring Comparison
Domain A (Series 1)
Domain B (Series 2)
Placed Students
70
65
60
55
50
45
40
35
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0
Academic Year


Student Expense Allocation
Books
Food
12.0%
18.0%
25.0%
Rent
45.0%
Tuition



2. A survey company collects voting response IDs and numerical factor scores from public polls for statistical evaluation. Write Python programs for the following requirements:

(a) Given a list of n voter choice IDs, identify all distinct IDs that appear strictly more than $n/3$ times in the list.

(b) From a collection of positive and negative numerical factor scores, find the maximum possible product that can be obtained by choosing any three values.

Solution:

```
# (a) Elements Appearing More Than N/3 Times
def elements_more_than_n_by_3(votes):
    n = len(votes)
    freq = {}
    for vote in votes:
        freq[vote] = freq.get(vote, 0) + 1
    result = [item for item, count in freq.items() if count > n // 3]
    return result
# (b) Maximum Product of Any Three Numbers
def max_product_of_three(nums):
    if len(nums) < 3:
```

```
        return 0
    nums.sort()
    # Case 1: Three largest positive numbers
    prod1 = nums[-1] * nums[-2] * nums[-3]
    # Case 2: Two most negative numbers (giving positive) * largest positive number
    prod2 = nums[0] * nums[1] * nums[-1]
    return max(prod1, prod2)
# Driver Code
voter_ids = [1, 2, 3, 1, 1, 2, 1, 4]
scores = [-10, -10, 5, 2, 8]
print("(a) Voter IDs with > n/3 votes:", elements_more_than_n_by_3(voter_ids))
print("(b) Maximum product of three numbers:", max_product_of_three(scores))
```

Output:

(a) Voter IDs with > n/3 votes: [1]

(b) Maximum product of three numbers: 800

3. A mechanical vibration laboratory analyzes natural frequencies and structural resonance modes of oscillating beams. Develop suitable Python programs using the NumPy linalg package for the following requirements:

(a) Given a symmetric stiffness matrix A, compute its eigenvalues and eigenvectors. Verify that the eigenvectors corresponding to distinct modes are mutually orthogonal by evaluating their dot product.

(b) For a given vibration transition matrix, obtain the eigenvalues and eigenvectors, and numerically demonstrate that $Av = \lambda v$ using np.allclose.

(c) Check whether a coordinate transformation matrix is singular by computing its determinant and verifying whether its inverse can be formed.

Solution:

```
import numpy as np
# (a) Symmetric Matrix and Orthogonal Eigenvectors
K = np.array([
    [3.0, 1.0],
    [1.0, 3.0]
])
```

```
vals, vecs = np.linalg.eig(K)
v1 = vecs[:, 0]
v2 = vecs[:, 1]
ortho_dot = np.dot(v1, v2)
print("--- (a) Orthogonality of Eigenvectors ---")
print("Eigenvalues:", vals)
print("Eigenvectors:\n", vecs)
print("v1 . v2 =", ortho_dot)
print("Are eigenvectors orthogonal?:", np.isclose(ortho_dot, 0.0))
# (b) Eigenvalues and Eigenvectors Verification
A_vib = np.array([
    [6.0, 3.0],
    [2.0, 7.0]
])
eigen_vals, eigen_vecs = np.linalg.eig(A_vib)
print("\n--- (b) Av = lambda*v Check ---")
for i in range(len(eigen_vals)):
    lam = eigen_vals[i]
    v = eigen_vecs[:, i]
    is_valid = np.allclose(np.dot(A_vib, v), lam * v)
    print(f"Eigenpair {i+1} [lambda = {lam:.2f}] satisfies Av = lambda*v: {is_valid}")
# (c) Singularity and Invertibility Test
T_singular = np.array([
    [1.0, 2.0, 3.0],
    [2.0, 4.0, 6.0],
    [0.0, 1.0, 2.0]
])
det_T = np.linalg.det(T_singular)
print("\n--- (c) Singularity Test ---")
print("Determinant:", det_T)
if np.isclose(det_T, 0.0):
```

```
    print("Matrix is singular: Inverse does not exist.")
else:
    print("Matrix is invertible.")
```

Output:

```
--- (a) Orthogonality of Eigenvectors ---
Eigenvalues: [4. 2.]
Eigenvectors:
 [[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
v1 . v2 = 0.0
Are eigenvectors orthogonal?: True

--- (b) Av = lambda*v Check ---
Eigenpair 1 [lambda = 4.00] satisfies Av = lambda*v: True
Eigenpair 2 [lambda = 9.00] satisfies Av = lambda*v: True

--- (c) Singularity Test ---
Determinant: 0.0
Matrix is singular: Inverse does not exist.
```